MiniOS — Modules 1 and 2

Integration, Experiments, Demonstration, Report and Viva

Kernel._.Crew CS2101 Operating Systems — Mini Project
Module 1 — Anvitha · Module 2 — Srujana
Modules 3–5 — Aarnavi, Advik, Krishna (upcoming)

8 October 2026

Contents

About this report **3**
Scope . . . 3
Status at the time of writing . . . 3
How to reproduce everything in this report . . . 4

Part 0 — Team, Scope and Comparison **4**
The team . . . 4
Scope, stated plainly . . . 4
What the handbook asks for, and what we handed in . . . 4
Commands the handbook does not ask for . . . 6
Beyond the code . . . 6
Three extra modes . . . 7
What we are not claiming . . . 7

Part I — Functionality and Integration **8**
1. What each module owns . . . 8
2. How they are joined . . . 8
3. What a typed word does now . . . 9
4. The three conflicts we had to resolve . . . 10
5. Build it . . . 11
6. Run it . . . 11
7. Prove it works . . . 12
8. Demonstrating it . . . 12
9. Adding Module 3, 4 or 5 . . . 12

Part II — Experiments and Analysis **14**
The machine . . . 14
Experiment 1 — What does fork cost, and what does exec add? . . . 14
Experiment 2 — Does fork() copy the parent’s memory? . . . 15
Experiment 3 — Zombie and orphan processes . . . 17
Experiment 4 — What does our own `processes` command cost? . . . 18
Experiment 5 — stdio buffering survives fork() . . . 18
Experiment 6 — MiniOS measuring itself . . . 19
Summary of findings . . . 20
Questions an examiner could ask about this section . . . 20

Part III — Project Demonstration **22**
1. Before you start . . . 22
2. Who says what . . . 22
3. The script . . . 22
4. The one-command fallback . . . 25
5. Questions likely to interrupt the demo . . . 25
6. If it breaks live . . . 26

Part IV — Project Report **27**
1. Aim and scope . . . 27
2. Architecture . . . 27
3. Module 1 — reading the machine through /proc . . . 28
4. Module 2 — creating processes . . . 30
5. Integration . . . 31
6. Testing . . . 31
7. Findings . . . 31
8. Limitations and what remains . . . 32
9. References . . . 32

Part V — Individual Viva Preparation **34**
A. The handbook’s 13-item checklist . . . 34
B. The handbook’s examiner questions . . . 37
C. Questions about our actual code . . . 37
D. The modules we have not built — honest answers . . . 38
E. Two-minute self-test before you walk in . . . 39
F. If something goes wrong in the viva . . . 39

About this report

This report accompanies the MiniOS source submission. It covers all five graded components.

Component	Weight	Where
Team, scope and comparison with the handbook	—	Part 0
MiniOS functionality and integration	45%	Part I
Experiments and analysis	15%	Part II
Project demonstration	20%	Part III
Report / documentation	10%	Part IV
Individual viva	10%	Part V

Part 0 names the team and sets out, command by command, what the handbook asks for and what we actually handed in.

Part I explains how Module 1 and Module 2 were merged into one program: what each module owns, where the two meet, the three conflicts that had to be resolved, how to build and run it, how to verify it, and how Modules 3 to 5 attach later.

Part II reports six experiments on process creation and `/proc`. Every number was measured on the machine described there, not quoted from a textbook. The programs that produced them are in `experiments/` and the raw output is in `docs/results.txt`.

Part III is the run sheet for the live demonstration: timings, the exact commands, the sentence to say for each one, and what to do when something fails in the room.

Part IV is the project report — design decisions, what each module does and why, testing method, findings, and an honest list of what is not built.

Part V is viva preparation: the handbook's own 13-item checklist and examiner questions answered, plus the code-level questions an examiner can ask with our source open in front of them.

Scope

This covers **Modules 1 and 2** (handbook Chapters 3 and 4). Modules 3 to 5 — pipes, threads, semaphores — and the optional xv6 system call are not merged yet; Part IV section 8 states exactly where that leaves the project, and Part V section D gives honest answers for the concepts they cover.

Status at the time of writing

```
build          clean, zero warnings
               -std=c11 -O2 -g -Wall -Wextra

test suite     34 checks, 34 passing
               ./tests/run_all.sh
```

```
experiments    6 experiments, all reproducible
               ./experiments/run_experiments.sh
```

How to reproduce everything in this report

```
make                                build everything
./tests/run_all.sh                  run the 34 checks
./experiments/run_experiments.sh    reproduce Part II
./docs/figures/make_chart.py        redraw Figure 7 from the raw data
./docs/make_pdf.sh                  rebuild this PDF
```

Part 0 — Team, Scope and Comparison

The team

Kernel._.Crew — CS2101 Operating Systems mini project.

Kernel._.Crew — who owns what

Each module is one command table. The two merged modules did not require any change to each other's code.

MODULE	HANDBOOK	CONCEPT	OWNER	STATUS
M1	Ch 3	/proc — system, cpu, top	**Anvitha**	MERGED
M2	Ch 4	run, processes	**Srujana**	MERGED
M3	Ch 5	pipes and IPC — submit		UPCOMING
M4	Ch 6–7	threads, race conditions	**Aarnavi · Advik Krishna**	UPCOMING
M5	Ch 7–8	semaphores, queue, xv6		UPCOMING

Figure 6 — Module ownership and project status. Modules 3–5 attach through the same one-line mechanism Module 2 used, with no change to any existing file.

Scope, stated plainly

This submission is **Modules 1 and 2**. Modules 3–5 are not merged, and nothing in this report claims otherwise. Part IV section 8 lists exactly what is missing; Part V section D gives honest answers for the concepts those modules cover, because the viva is individual.

What the handbook asks for, and what we handed in

The handbook specifies five commands for Chapters 3 and 4 and marks seven TODOs. All seven are filled. Everything in the right-hand column beyond that is ours.

`system` — handbook section 3.3

The handbook's skeleton is four TODOs:

```c
void show_system_info(void)
{
    // TODO 1: open /proc/uptime
    // TODO 2: read the required value
    // TODO 3: read one memory value
    // TODO 4: display them clearly
}
```

Handbook asks for	We produce
uptime	uptime **and** idle time, with a human-readable duration
one memory value	7 keys from `/proc/meminfo`: total, free, available, buffers, cached, swap total, swap free
—	a usage bar and a used-percentage
—	load averages and runnable/total task counts from `/proc/loadavg`
—	host, kernel release and architecture via `uname`
—	our own PID, so the examiner can go and read `/proc/<pid>/status`
"display them clearly"	aligned columns, colour when the terminal supports it

`run` — handbook section 4.6

Three TODOs: create a child, exec, parent waits. All three filled.

Handbook asks for	We produce
fork → exec → waitpid	the same, plus the three defences below
—	`fflush(NULL)` before `fork`, so buffered output is not duplicated
—	`_exit` not `exit` in the child, so inherited buffers are not flushed twice
—	`waitpid` retried on `EINTR`, so Ctrl-C does not abandon the child
—	exit status decoded, including **signal** deaths as 128 + signal
—	wall-clock timing per child, which makes `run` its own benchmark
handbook 4.5: "print the exit status"	done, and distinguishes exit 127 (exec failed)

`processes`

Handbook asks for	We produce
“observing processes”	PID, PPID, state, thread count, RSS, CPU seconds, name
—	`-m` sort by memory, `-c` sort by CPU
—	`-n N` limit rows
—	`-t` only our own children — which is how you show you leave no zombies
—	state letters translated (`R` → running, `Z` → zombie, …)
—	a summary line counting each state

`help` and `quit`

Handbook asks for	We produce
`help`	`help` grouped by module, **and** `help <command>` for one command’s usage
`quit`	`quit` and `exit`, with the terminal restored on every exit path

Commands the handbook does not ask for

Both have to justify themselves against the handbook’s own warning — *“Do not make MiniOS huge… Which OS concept does this feature teach?”*

Command	The concept it teaches
`cpu`	A monotonic counter is meaningless alone. `/proc/stat` counts ticks since boot; the usage *now* only exists as the difference between two readings.
`top`	Three mechanisms at once: ANSI redraw, `termios` raw mode, and `select()` waiting on *either* a timer *or* stdin — the alternative is a loop that ignores the keyboard or busy-waits.

Beyond the code

Handbook’s starter package	What we handed in
`tests/test_m1.sh`, `tests/test_m2.sh`	one suite, **34 checks**, including 3 that test the module seam itself

Handbook's starter package	What we handed in
`examples/` — 3 relevant demo programs	the same 3, still compiling, plus 2 load generators used by the experiments
no experiments asked for in Ch 3–4	**6 experiments**, 405 lines, with medians over repeated runs
report	5-part report: integration, experiments, demonstration, design, viva
—	every figure and number in it regenerates from the raw data

Three extra modes

Not in the handbook, but they are what make the demo and the test suite possible:

```
./minios                 interactive
./minios -c "system"     run one command and exit
./minios demo.txt        run a script of commands, echoing each
./minios --version
```

What we are not claiming

- Modules 3–5 do not exist in this build.
- Module 2's own extensions (`ps`, `wait`, `kill`, `runlimit`, `limit`) were listed in its command table but their source files were not handed over. Those five rows are commented out, not deleted — see Part I section 4.2.
- The experiments cover Chapters 3–4 only. The handbook's week 8 ("producer-consumer and performance") needs Modules 4 and 5.
- Thermal sensor readings are untested: this machine has no sensors. The zero-sensor path is exercised; the reading path is not.

Part I — Functionality and Integration

How Module 1 and Module 2 became one program. Graded component: 45%.

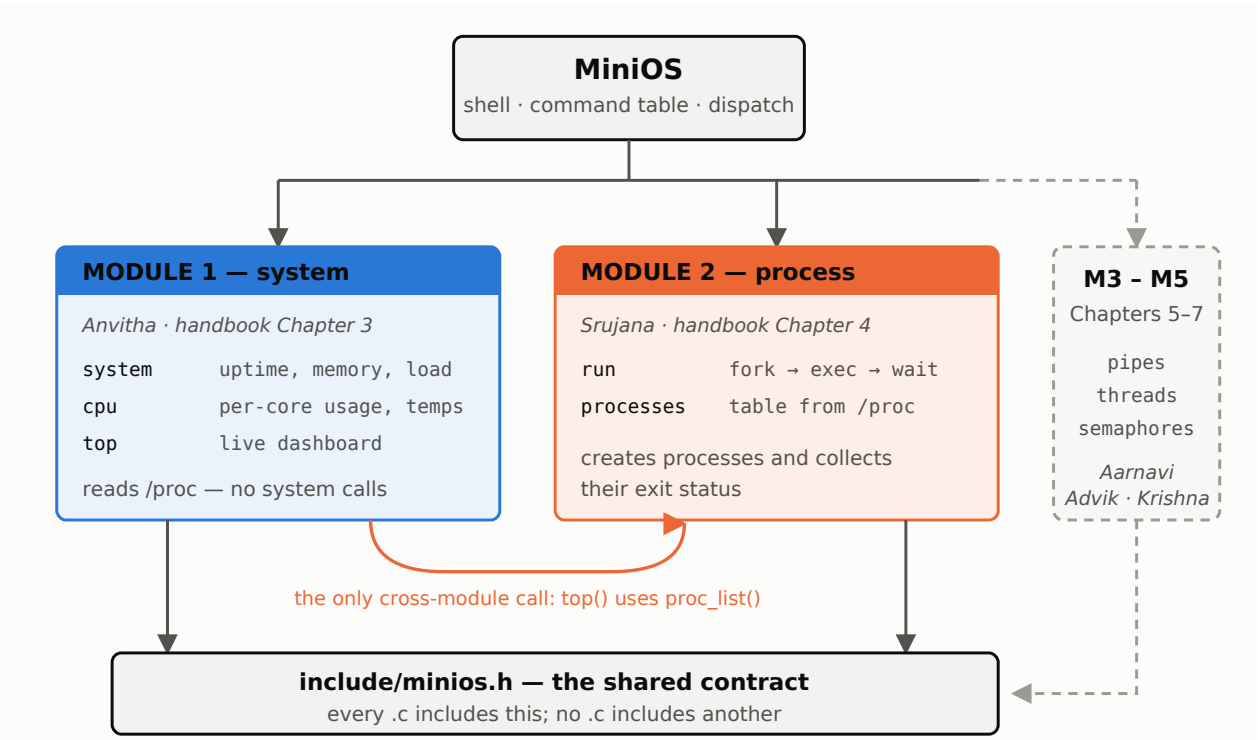


Figure 1 — MiniOS architecture and module ownership.
Modules meet only at the shared header. Modules 3–5 attach the same way, with no change to existing code.

1. What each module owns

```
MODULE 1  (Anvitha)                MODULE 2  (Srujana)
handbook Chapter 3                 handbook Chapter 4
--------------------               --------------------
src/shell.c     prompt + dispatch  src/process.c    run, processes
src/util.c      helpers
src/monitor.c   system             fork() -> exec() -> waitpid()
src/cpustat.c   cpu
src/dashboard.c top

reads:  /proc/uptime               reads:  /proc/<pid>/stat
        /proc/meminfo
        /proc/loadavg
        /proc/stat
```

Nobody edits anybody else's `.c` file. That is the whole rule.

2. How they are joined

One header, one list. Nothing else.

```
          src/shell.c          src/process.c
```

```
|                     |
|  both #include      |
+----------+----------+
           v
  include/minios.h
   (the contract)
```

`minios.h` says *what exists*. Each `.c` file says *how it works*. Neither module includes the other's source.

The seam, in two places

A. `include/minios.h` — one line per module's table:

```
extern const command_t cmds_shell[];     /* M1 */
extern const command_t cmds_process[];   /* M2 */
```

B. `src/shell.c` — one row per module:

```
static const command_t *const all_tables[] = {
    cmds_shell,      /* M1 — Anvitha */
    cmds_process,    /* M2 — Srujana */
    NULL
};
```

That is the entire integration. Two lines of real change.

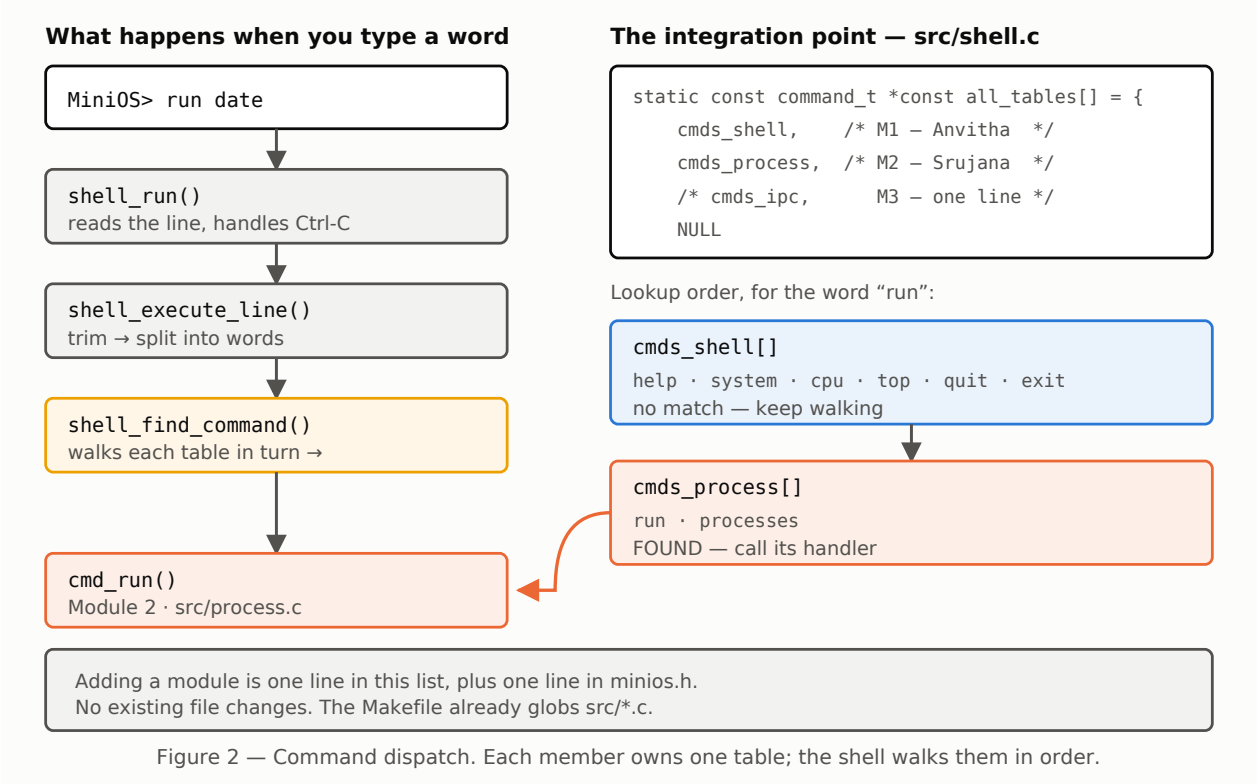


Figure 2 — Command dispatch. Each member owns one table; the shell walks them in order.

3. What a typed word does now

```
you type:  run date
```

```
    |
    v
shell_run()              reads the line
    |
    v
shell_execute_line()     trim -> split into words
    |
    v
shell_find_command()     walks table 1 (M1): help system cpu top quit
    |                    not found
    |                    walks table 2 (M2): run processes
    |                    FOUND
    v
cmd_run()                in process.c — M2's code
    |
    v
run_program()            fork -> exec -> waitpid
```

Before the merge it stopped after table 1. After the merge it keeps walking. That is it.

4. The three conflicts we had to resolve

Honest list. These are real things that broke.

4.1 Both modules defined `processes`

M1's standalone build had its own `src/procinfo.c` with `proc_read`, `proc_list` and `cmd_processes`. M2's `process.c` has the same three functions.

```
BEFORE                          AFTER
procinfo.c  proc_list()         procinfo.c   DELETED
process.c   proc_list()         process.c    proc_list()   <- kept
-> duplicate symbol                -> one owner
```

Decision: M2 keeps it. Chapter 4 is their chapter. `src/procinfo.c` was deleted and the `processes` row was removed from M1's table in `shell.c`.

M1's `top` now calls M2's `proc_list()`. That is not a workaround — that is the integration working.

4.2 Five commands with no code behind them

`process.c` arrived with a 7-row command table. Only 2 of the 7 handlers are in the file:

```
run         cmd_run         -> defined in process.c   OK
processes   cmd_processes   -> defined in process.c   OK
ps          cmd_ps          -> bgjobs.c    NOT HANDED OVER
wait        cmd_wait        -> bgjobs.c    NOT HANDED OVER
```

```
kill       cmd_kill       -> bgjobs.c    NOT HANDED OVER
runlimit   cmd_runlimit   -> rlimit.c    NOT HANDED OVER
limit      cmd_limit      -> rlimit.c    NOT HANDED OVER
```

Linking with all 7 gives five `undefined reference` errors.

Decision: the five rows are commented out, not deleted. See the marked block in `src/process.c`. To restore:

1. drop `bgjobs.c` and `rlimit.c` into `src/`
2. uncomment the prototypes at the bottom of `include/minios.h`
3. uncomment the five rows in `src/process.c`
4. `make`

Nothing else changes. The Makefile picks up new `.c` files on its own.

4.3 A symbol in the wrong file

`minios_interrupted` (the Ctrl-C flag) was defined in `shell.c`, but `util.c` reads it inside `sleep_ms()`. Any tool that links `util.c` without `shell.c` failed to link.

Fix: moved the definition into `util.c`, next to the code that uses it. `shell.c` still sets it from the signal handler.

5. Build it

```
make              everything           -> ./minios, build/ex_*, build/exp*
make examples     just the handbook demos
make experiments  the same, then tells you how to run them
make clean        start over
```

Flags: `-std=c11 -O2 -g -Wall -Wextra -D_GNU_SOURCE`

Builds with **zero warnings**. That is deliberate — a warning in this codebase means something is actually wrong.

6. Run it

```
./minios                  interactive
./minios demo.txt         run a script of commands
./minios -c "system"      run one command
./minios --version
```

The commands

```
MODULE 1
  help [command]              list commands, or explain one
  system                      uptime, memory, load  (Ch 3)
  cpu [sample_ms]             per-core usage, temperatures
  top [interval] [frames]     live dashboard, q to quit
```

```
MODULE 2
  run <program> [args]        fork -> exec -> wait  (Ch 4)
  processes [-m|-c] [-n N] [-t]  process table from /proc

  quit / exit
```

7. Prove it works

```
./tests/run_all.sh
```

34 checks. Every command in section 6, plus the four handbook example programs, plus three checks that specifically test the seam between the modules:

```
PASS  top uses M2's proc_list
PASS  both tables reachable
PASS  script mode runs mixed cmds
```

Last run: **34 passed, 0 failed.**

8. Demonstrating it

The full run sheet — timings, the exact commands, what to say for each one, and what to do if something fails in the room — is **Part III**.

The short version, if you only need the order:

```
./minios
help -> system -> cpu 500 -> run date -> run nosuchthing
     -> processes -m -n 10 -> processes -t -> top (q) -> quit
```

No typing at all:

```
./minios demo.txt
```

9. Adding Module 3, 4 or 5

The seam is designed for this. For a new module `foo`:

```
1. write src/foo.c, ending with:

       const command_t cmds_foo[] = {
           { "bar", cmd_bar, "bar [n]", "what it does" },
           { NULL, NULL, NULL, NULL }
       };

2. add to include/minios.h:

       extern const command_t cmds_foo[];
       cmd_status_t cmd_bar(int argc, char **argv);

3. add to the list in src/shell.c:
```

```
    cmds_foo,     /* M3 */
```

```
4. add the label to owners[] in cmd_help()
```

```
5. make
```

No other file is touched. No Makefile edit — it globs `src/*.c`.

Two rules for the team:

- You may **add** to `minios.h`. You may not **change or delete** a line someone else added — their `.c` file depends on it.
- Two modules must never claim the same command word. The first table in the list wins, silently, which is a horrible bug to find.

Part II — Experiments and Analysis

Six experiments. Every number below was measured, not looked up. Graded component: 15%.

Reproduce everything:

```
make
./experiments/run_experiments.sh
```

Raw output is in `docs/results.txt`.

The machine

```
kernel    Linux 6.18.44
cpus      2
memory    8031 MB
compiler  gcc 13.3.0
flags     -std=c11 -O2 -g -Wall -Wextra
```

Absolute times are machine-specific. Run it on your own laptop and the milliseconds will differ. The *ratios* are the result; they held across every repeat we did.

Experiment 1 — What does fork cost, and what does exec add?

Question. The handbook treats `fork()` and `exec()` as two separate steps. How separate are they, in cost?

Method. 300 iterations each, after a warm-up round.

```
A.  fork + _exit + wait      just making a process
B.  fork + exec + wait       making it AND loading /bin/true
B - A                        the price of exec alone
```

Result.

Each benchmark is run 7 times and the **median** is reported. A single timing on a shared machine is mostly noise; one unlucky run cannot move the middle value, but it would wreck a mean.

what	median	range over 7 reps
fork + _exit + wait	0.125 ms	0.117 – 0.149
fork + exec + wait	1.130 ms	1.088 – 1.399
exec alone	**1.005 ms**	—

```
exec makes it slower by   9.04x     (worst case 11.98, best case 7.31)
```

Analysis. Making a process is the cheap half. **exec costs about 9x the fork that precedes it**, and stayed above 7x in every rep.

That ordering is not obvious, and it is worth being able to explain:

- `fork()` duplicates bookkeeping — a task struct, page tables marked read-only. No program is read from disk.
- `exec()` does the heavy work — find the file, check permissions, tear the old address space down, map the new ELF, resolve the dynamic linker, load `libc`, run the relocations, then jump to `_start`.

Why the spread is wide. This is a 2-CPU shared VM, so scheduling noise dominates the absolute numbers. The ratio stayed above 7x in every rep (worst 11.98x, best 7.31x), and that range — not the single median — is the claim we are willing to defend.

Experiment 2 — Does fork() copy the parent's memory?

Question. Handbook 4.3 shows parent and child have independent variables. The obvious conclusion is that fork copies everything. Is it true?

Method. Parent allocates `N` MB and writes one byte in every 4096-byte page, so the pages genuinely exist. Then time fork+wait, 40 rounds.

Two cases:

```
case A   child calls _exit(0) immediately — never touches the memory
case B   child writes one byte per page — forces every page to be real
```

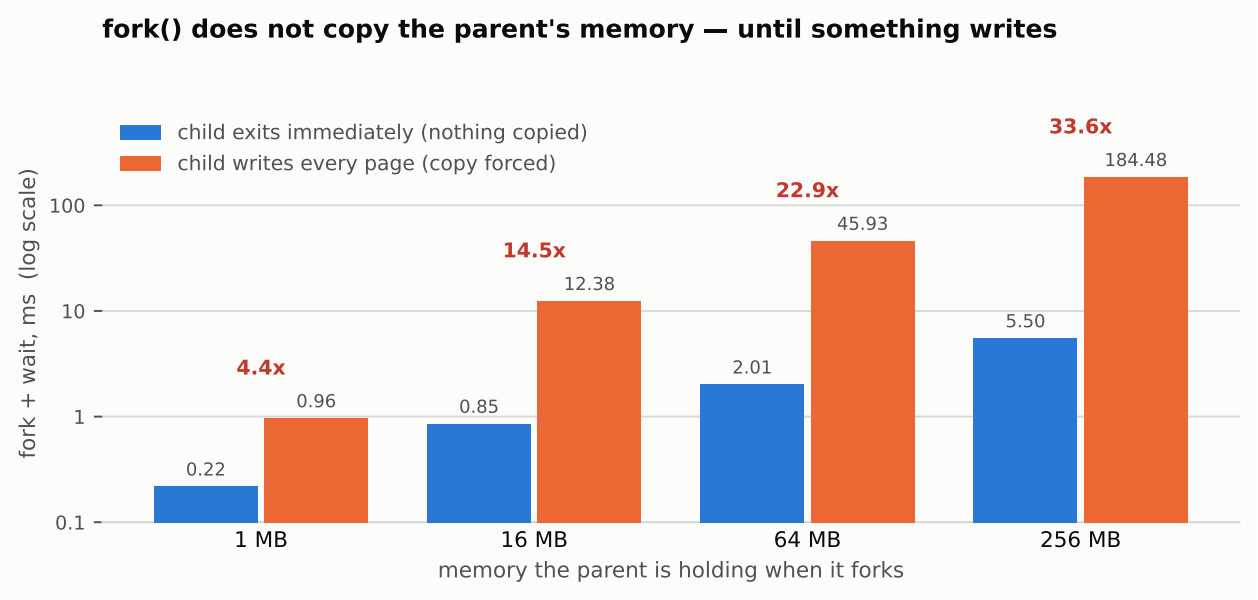
Result.

parent holds	A: child exits	B: child writes	B / A
1 MB	0.22 ms	0.96 ms	4.4x
16 MB	0.85 ms	12.38 ms	14.5x
64 MB	2.01 ms	45.93 ms	22.9x
256 MB	5.50 ms	184.48 ms	**33.6x**

Per megabyte, at 256 MB:

```
case A   0.0215 ms / MB
case B   0.7206 ms / MB      <- 34x more
```

Note the ratio *grows* with size (4.4x → 33.6x). At 1 MB the fixed cost of fork itself dominates; by 256 MB the page copying dominates. That trend is itself evidence for the explanation below.



Analysis. The answer is **no — fork does not copy the memory.**

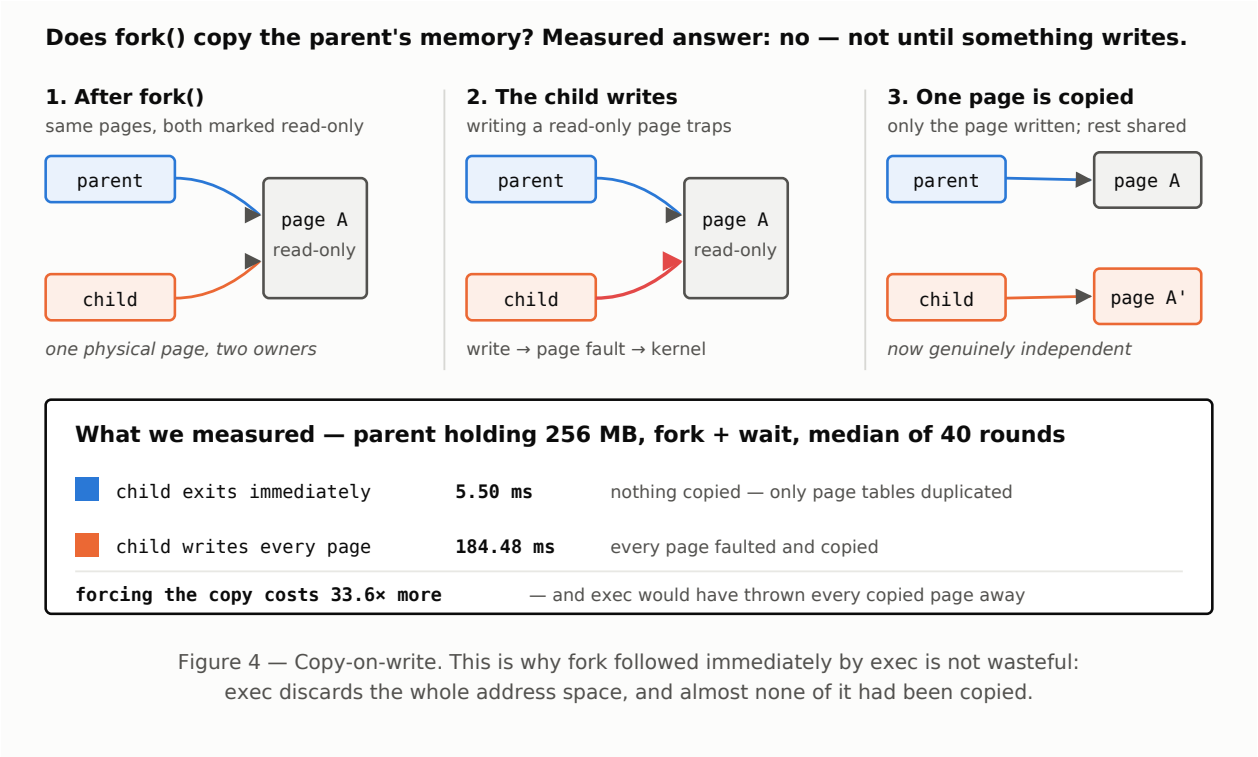


Figure 4 — Copy-on-write. This is why fork followed immediately by exec is not wasteful: exec discards the whole address space, and almost none of it had been copied.

```
fork()
  |
  +-- page tables duplicated, every page marked READ-ONLY
  |   both processes point at the SAME physical pages
  |
  +-- child writes to a page
        |
        v
      hardware fault  ->  kernel copies THAT ONE PAGE  ->  resume
```

This is **copy-on-write**. The copy is deferred until somebody writes, and only the

written page is copied.

But case A is not flat, and that matters. 1 MB costs 0.22 ms, 256 MB costs 5.50 ms — 25x more for 256x the memory. So fork is not free even when nothing is copied: the kernel still has to walk and duplicate the page *tables*. 256 MB is 65,536 pages, and each needs an entry. That work is proportional to the number of pages, just ~34x cheaper than copying their contents.

Why this makes `run` fast. When you type `run date`, the child immediately calls `exec`, which throws the whole inherited address space away. Every page copied would have been wasted. Copy-on-write means almost none were. That is the entire reason the fork+exec pair is a sensible design and not an obviously wasteful one.

A note on method. The first version of this experiment used `memset()` in the child and produced nonsense — 256 MB appeared *faster* than 1 MB. At `-O2`, gcc saw that nothing read the buffer afterwards and deleted the loop. The fix was `volatile char *`. The lesson is in the code comment, and it is a fair thing to be asked about: **an optimiser can delete the thing you are trying to measure.**

Experiment 3 — Zombie and orphan processes

Question. What happens when the parent misbehaves?

Method. Create each condition deliberately, then read the state letter and PPID back out of `/proc/<pid>/stat`.

Result.

```
ZOMBIE — child exits, parent does not wait()

  before wait()     pid=3841  state=Z  ppid=3840     <- ZOMBIE
  after  wait()     pid 3841 is gone from /proc


ORPHAN — parent exits first, child keeps running

  grandchild now    pid=3844  state=S  ppid=1        <- REPARENTED
```

Analysis.

Zombie. The child is finished — its memory is gone, it will never run again. But its exit status has to be stored somewhere, because the parent has not asked for it yet. The kernel keeps a stub entry in the process table. State `Z`. The entry vanishes the instant `waitpid()` collects it.

A zombie is not a leak of memory. It is a leak of **PID table entries** — and the table is finite, so a server that forks in a loop and never waits eventually cannot fork at all.

Orphan. The child outlives its parent and would have PPID pointing at nothing. Linux reparents it to PID 1. Observed: `ppid=1`. init calls `wait()` forever, so the orphan gets reaped when it finally exits and never becomes a permanent zombie.

Tie-back to our code. `run_program()` in `process.c` calls `waitpid` in a loop that retries on `EINTR`. That loop is why `run` never leaves a zombie behind. Verify it live:

```
MiniOS> run /bin/true
MiniOS> processes -t        # no Z rows
```

Experiment 4 — What does our own `processes` command cost?

Question. `top` calls `proc_list()` once per frame. Is that affordable at a fast refresh rate?

Method. Links against the project's real `process.o` — this measures *our* implementation, not a rewrite. 100 full scans.

Result.

```
processes on this machine        64
one full /proc scan              0.413 ms   (median of 7 batches)
cost per process                 0.0065 ms
scans possible per second        2422
```

Projected cost of running `top` continuously:

refresh	CPU used by the scan
2.0 s (default)	0.021 %
0.1 s	0.413 %

Analysis. 6.5 microseconds per process. A full scan costs about **3x what a bare `fork()` costs** (0.413 ms vs 0.125 ms) — a useful sense of scale: reading the entire process table is the same order of magnitude as creating one process.

The cost is linear in process count, because each process means one `open` + `read` + `close` on `/proc/<pid>/stat`. On a busy server with 2000 processes this would be ~13 ms per scan, and a 0.1 s refresh would start to cost real CPU. At 64 processes it is free.

Design conclusion. The 2-second default refresh is justified by the measurement, not by taste. We could refresh 10x faster and still spend under 0.5% of one core.

Experiment 5 — stdio buffering survives fork()

Question. Run the handbook's `fork_demo` twice — once to the terminal, once through a pipe. Same program, same input. Same output?

Result.

```
to a terminal                    through a pipe
---------------                  ----------------
Before fork                      Before fork
I am the parent. Child PID=16589 I am the parent. Child PID=16601
I am the child.  PID=16589       Before fork            <- again!
```

```
                                  I am the child.  PID=16601

"Before fork" printed 1 time      "Before fork" printed 2 times
```

(The *order* of the parent and child lines varies between runs — they are two processes racing. The *count* does not, and the count is the result.)

Analysis. The program has exactly one `printf("Before fork\n")`. It appears twice. Nothing forked twice — `fork()` copied a *buffer*.

```
to a terminal                  through a pipe
stdout is line-buffered        stdout is fully buffered

printf -> flushed at '\n'      printf -> sits in the buffer
          buffer EMPTY                   buffer HOLDS "Before fork\n"
             |                              |
           fork()                         fork()
             |                              |
       child gets empty buffer        child gets a COPY of the
             |                        buffer, still holding the text
       child exits, nothing extra           |
                                      child exits -> flushes its copy
                                            -> prints it a second time
```

Why it matters for MiniOS. This is a real bug class, not a curiosity. `run_program()` in `process.c` calls:

```
fflush(NULL);              /* otherwise buffered output is duplicated */
pid = fork();
```

That one line is the fix. Delete it and `./minios script.txt > out.txt` starts duplicating lines, while `./minios` interactively looks perfectly fine — the worst kind of bug, because it only appears when redirected.

Also note the child uses `_exit(127)` and not `exit(127)` after a failed exec. `exit()` would flush the inherited stdio buffers a second time; `_exit()` skips that.

Experiment 6 — MiniOS measuring itself

`run` reports wall-clock time for each child, so the shell can be its own stopwatch.

```
[run] /bin/true          exited with status 0 after    2.6 ms
[run] /bin/date          exited with status 0 after    1.3 ms
[run] ex_hog_cpu 0.2     exited with status 0 after  207.1 ms
```

Analysis. `/bin/true` does nothing at all and still costs 1–3 ms — that is pure fork+exec overhead, and it is the same order as Experiment 1 (1.13 ms) measured by a completely separate program. Two independent measurements agreeing is the useful part. (`run` is a single untimed sample, so it sits at the noisy end of the Exp 1 range rather than at the median.)

`hog_cpu 0.2` was asked for 0.2 s of CPU and took 207.1 ms wall-clock. The ~7 ms gap is fork+exec overhead plus scheduling. On a loaded machine wall-clock would exceed CPU time by much more, because the process would spend time in the run queue not running — which is exactly the distinction `system` shows between uptime and idle time.

Summary of findings

#	question	finding
1	fork vs exec cost	exec is ~9x the cost of the fork before it (never below 7x)
2	does fork copy memory	no — copy-on-write; forcing the copy costs 34x more
2b	is fork free then	no — page-table setup is still linear in page count
3	zombie	exists until `wait()`; leaks PID slots, not memory
3b	orphan	reparented to PID 1, reaped normally
4	cost of our /proc scan	6.5 us per process; `top` uses 0.021% CPU
5	buffering across fork	duplicated output when piped; `fflush(NULL)` fixes it
6	self-measurement	same order as Exp 1, from independent code

The one sentence version. `fork()` is cheap because it copies almost nothing; `exec()` is the expensive step; and the one thing `fork()` does copy that people forget about is the stdio buffer.

Questions an examiner could ask about this section

- *Why is exec slower than fork?* — fork duplicates bookkeeping; exec reads a file from disk, builds a new address space and runs the dynamic linker.
- *If fork doesn't copy memory, why do parent and child see different values?* — the pages are shared read-only; the first write faults and the kernel copies that single page. Independence is preserved, the copying is just deferred.
- *Your Experiment 2 case A still grows with size. Doesn't that disprove copy-on-write?* — no, it measures page-table duplication, which is proportional to page count. Case B is 31x more expensive, which is the content copy.
- *Why did your first version of Experiment 2 give nonsense?* — `-O2` dead-store elimination removed the child's write loop. Fixed with `volatile`.

- *Why does run call `fflush(NULL)`?* — Experiment 5. Without it, buffered output is duplicated into the child.
- *How many zombies can you create before something breaks?* — until the PID table is exhausted; see `/proc/sys/kernel/pid_max`.
- *Why report a median and not an average?* — one scheduling hiccup on a shared VM adds a large outlier. It moves a mean a lot and a median not at all.
- *Would `top` be usable on a machine with 2000 processes?* — the scan is linear, so ~10 ms per frame. Fine at 2 s refresh, questionable at 0.1 s.

Part III — Project Demonstration

The run sheet for the live demo. Graded component: 20%.

Eight minutes, rehearsed. The aim is not to show every feature — it is to show that we know **which OS idea each command demonstrates**.

1. Before you start

Run this the morning of, not five minutes before.

```
cd minios-m1-m2
make clean && make          expect: no warnings at all
./tests/run_all.sh          expect: 34 passed, 0 failed
./minios demo.txt           expect: runs to the end, no errors
ls build/exp1_fork_vs_exec  expect: it exists (the demo runs it)
```

Then set the room up:

```
[ ] terminal at least 100 x 30, or top's bars wrap and look broken
[ ] font size up — the examiner is reading from across a desk
[ ] clear the screen; no editor, no other tabs
[ ] cd into the project directory and stay there
[ ] close anything heavy; a loaded machine makes 'cpu' look odd
[ ] have docs/MiniOS_Evaluation.pdf open on a second window
```

2. Who says what

Minutes	Part	Presenter
0:00–0:30	What MiniOS is, the architecture diagram	M1
0:30–3:00	Module 1 — `system`, `cpu`, `top`	M1
3:00–5:00	Module 2 — `run`, `processes`	M2
5:00–6:00	Integration — `help`, the table list, `top` using M2	M1
6:00–7:30	Experiments — Exp 1, 2 and 5	either
7:30–8:00	What is not built yet, and the plan	M1

Both of you should be able to deliver all of it. The examiner may swap you deliberately.

3. The script

Each step: the command, then the one sentence that earns the mark. Say the sentence **while** the output is on screen.

Opening (30 s)

`./minios`

> “MiniOS is a small control room for the machine. Two modules are merged so far — Module 1 reads the system through /proc, Module 2 creates and watches processes. Everything you’ll see is ordinary C calling ordinary system calls.”

Module 1 (2 min 30 s)

`help`

> “Commands are grouped by module. Each module owns one table, and the shell walks the tables in order. Adding Module 3 is one line.”

`system`

> “All of this came out of /proc. Not a system call — we opened three text files that don’t exist on disk. The kernel writes them when you read them.”

If asked where: `/proc/uptime`, `/proc/meminfo`, `/proc/loadavg`.

`cpu 500`

> “Here’s the one thing worth remembering. /proc/stat holds counters that only go up, since boot. A single reading is the average since boot, which is useless. We take two readings 500 ms apart and subtract — and *that* is the usage right now.”

Point at the formula if they want it:

```
busy% = 100 * (change in total − change in idle) / change in total
```

`top`

> “Same two-reading trick, repeated every two seconds. The bars are redrawn with ANSI escape codes, the terminal is in raw mode so `q` is picked up immediately, and `select()` waits for either the timer or a keypress — so it’s responsive without spinning the CPU.”

Let it refresh **twice**, press `+` once to show the interval change, then `q`.

> “Note the MiniOS row — it’s pinned even when it’s idle, so you can always see the process you’re talking to.”

Module 2 (2 min)

`run date`

> “Three system calls. `fork` makes a second process — it does *not* load a program. `exec` loads `date` into that process — it does *not* make one. `waitpid` collects the exit status. The timing is ours.”

`run definitelynotreal`

> "Status 127. `exec` failed, but the child already existed — so it still had to die, with `_exit`, not `exit`. If we'd used `exit` it would have flushed the buffers it inherited from the parent and we'd see duplicate output."

`processes -m -n 10`

> "One line of /proc//stat per process. 52 fields; we parse six. The name is in brackets and can contain brackets itself — `(Web Content)` is real — so we search for the *last* close bracket, not the first."

`processes -t`

> "Only our own children. Right now that's just MiniOS, because `run` waits properly and leaves no zombies behind."

Integration (1 min)

> "The two modules never include each other. They share one header."

Show `include/minios.h` and `src/shell.c:45`:

```
static const command_t *const all_tables[] = {
    cmds_shell,      /* M1 */
    cmds_process,    /* M2 */
    NULL
};
```

> "And they genuinely touch: `top` is Module 1, but it calls Module 2's `proc_list()` once per frame. Module 1 can't list processes on its own any more — we deleted that code deliberately so there's one owner."

`./tests/run_all.sh`

> "34 checks, including three that specifically test the seam."

Experiments (1 min 30 s)

Pick these three. Do not run all six live — too slow.

`./build/exp1_fork_vs_exec`

> "exec costs about nine times the fork before it. The second step is the expensive one, which surprised us — exec reads a file from disk and runs the dynamic linker; fork just duplicates bookkeeping."

`./build/exp2_cow`

> "This answers 'does fork copy the memory?'. No. When the child exits straight away, holding 256 MB costs 5.5 ms. When the child writes to every page and forces the copy, the same fork costs 184 ms — 34 times more. That's copy-on-write."

> "And it's why `run` is fast: the child calls exec immediately and throws the whole address space away, so almost nothing was ever copied."

```
./build/ex_fork_demo
./build/ex_fork_demo | cat
```

> "Same program, run twice. Piped, 'Before fork' prints twice. The program only prints it once — `fork` copied the *buffer*. A terminal is line-buffered so it's already flushed; a pipe isn't. That's why `run_program` calls `fflush(NULL)` before forking."

Closing (30 s)

> "Modules 3 to 5 — pipes, threads, semaphores — aren't merged yet. The framework for them is done: a new module is one table, one line in the header, one line in the shell. Our Chapter 5 to 7 experiments are planned but need that code."

Do not oversell. An honest boundary reads better than a vague claim.

4. The one-command fallback

If anything goes wrong, or you are short on time:

```
./minios demo.txt
```

Runs every command non-interactively, echoing each one. Works with no typing and no terminal tricks.

5. Questions likely to interrupt the demo

Short answers, so you can carry on.

Question	Answer in one breath
"Is that real data?"	Yes — compare with `uptime` / `free -m` / `htop` in another terminal.
"Why no temperatures?"	This machine has no `/sys/class/thermal` sensors. Normal in a VM or WSL. The code handles zero sensors.
"Why is cpu0 at 0%?"	500 ms is a short window on an idle machine. `cpu 2000` gives a longer one.
"Did you write this or copy it?"	Offer a line. `src/process.c:100` — the last-bracket parse — is the one to explain.
"What happens on Ctrl-C?"	Handler sets a flag only. `fgets` returns `EINTR`, we print a newline and reprompt. No `SA_RESTART`, on purpose.
"Show me a zombie."	`./build/exp3_zombie_orphan` — state `Z` before `wait()`, gone after.

6. If it breaks live

```
top looks frozen          -> you're in raw mode; press q. If stuck, Ctrl-C.
terminal echo is gone     -> type  reset  and press Enter (blind). It works.
bars wrap / look wrong    -> window too narrow. Widen, rerun.
'cpu' shows 0% everywhere -> idle machine, short window. Try  cpu 2000.
a command errors          -> say what you expected, then ./minios demo.txt
build fails on their box  -> make clean && make. If it still fails, show
                             docs/results.txt from a known-good run.
```

The rule: if something fails, say what *should* have happened and move on. Do not debug live. You have eight minutes.

Part IV — Project Report

Design, implementation and findings. Graded component: 10%.

1. Aim and scope

MiniOS is a small command-driven program for inspecting and controlling a Linux machine, built to make operating-system mechanisms visible rather than to be useful software. The handbook's framing — *"I know why this part exists, and I know which OS concept it demonstrates"* — was the test applied to every feature.

This report covers the two modules merged so far:

Module	Handbook	Concept	Commands
1	Chapter 3	the `/proc` filesystem	`system`, `cpu`, `top`
2	Chapter 4	process creation	`run`, `processes`

Modules 3–5 (pipes, threads, semaphores) and the optional xv6 system call are not merged. Section 8 says where that leaves the project.

2. Architecture

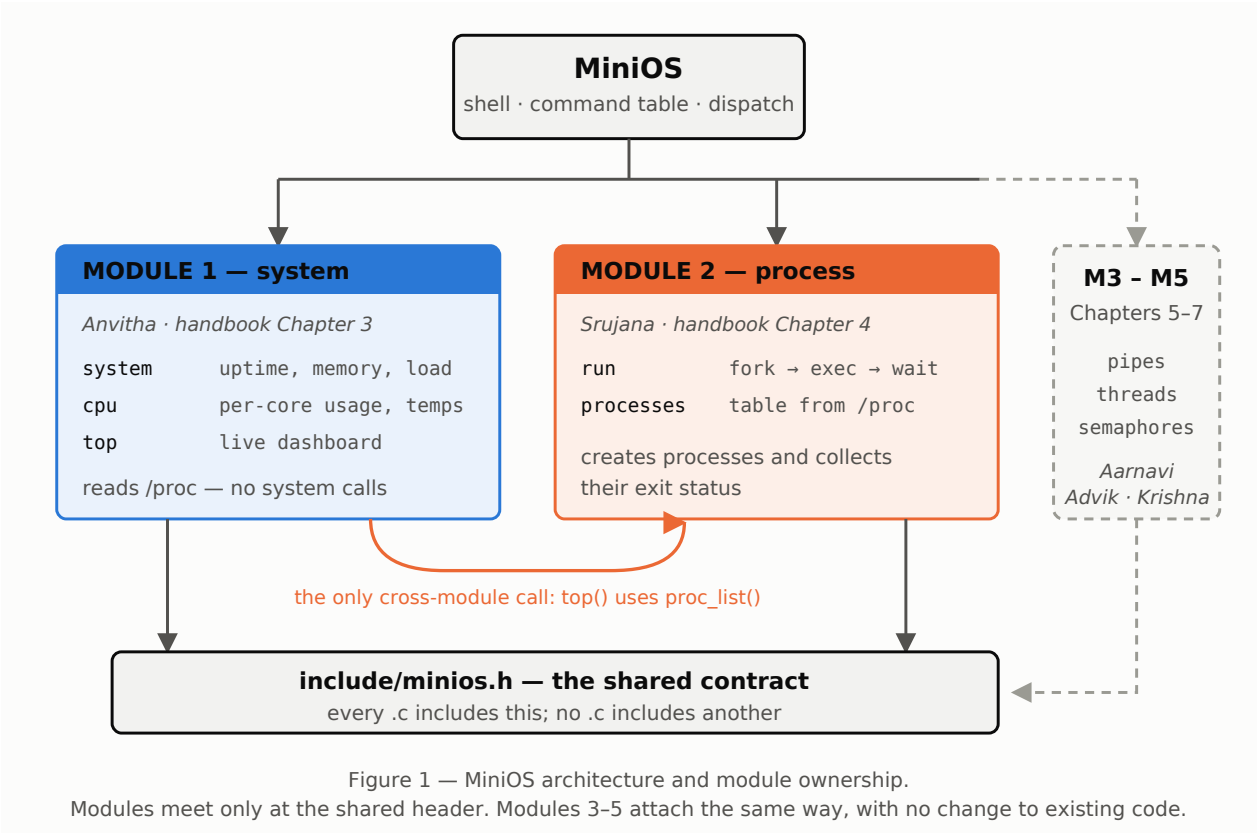
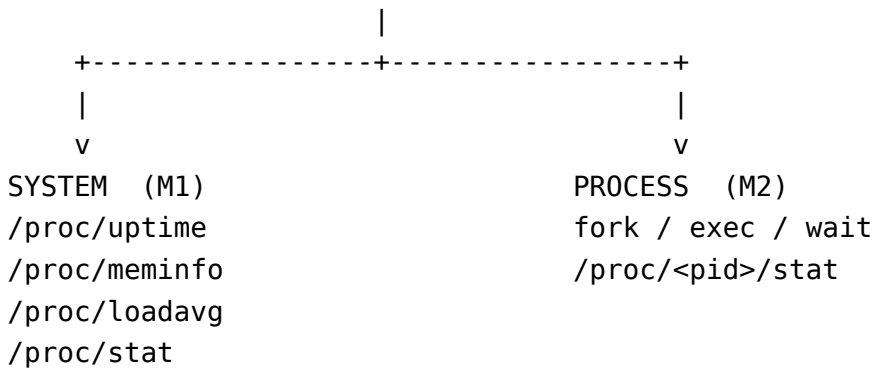


Figure 1 — MiniOS architecture and module ownership.
Modules meet only at the shared header. Modules 3–5 attach the same way, with no change to existing code.

```
+------------------+
|      MiniOS      |
|   "control room" |
+---------+--------+
```



Every `.c` file includes `include/minios.h` and nothing from any other `.c` file. The header declares, the sources implement. This is what makes five people able to work without blocking each other, and it is described in full in Part I.

Code size

File	Lines	Owner	Purpose
`include/minios.h`	167	shared	types and prototypes
`src/main.c`	60	Anvitha	start-up, argument handling
`src/shell.c`	187	Anvitha	prompt, command tables, dispatch
`src/util.c`	193	Anvitha	parsing, formatting, bars
`src/monitor.c`	138	Anvitha	`system`
`src/cpustat.c`	207	Anvitha	`cpu`
`src/dashboard.c`	318	Anvitha	`top`
`src/process.c`	224	Srujana	`run`, `processes`
total	**1494**		

Plus 97 lines of tests and 405 lines of experiment code.

3. Module 1 — reading the machine through /proc

3.1 The idea

`/proc` is not storage. The kernel generates the text at the moment of the read, which means kernel state can be inspected with `fopen` and `fscanf` and nothing else. Handbook Chapter 3 builds to one function, `show_system_info()`, with four TODOs: open `/proc/uptime`, read a value, read one memory value, display them. That is implemented in `src/monitor.c`.

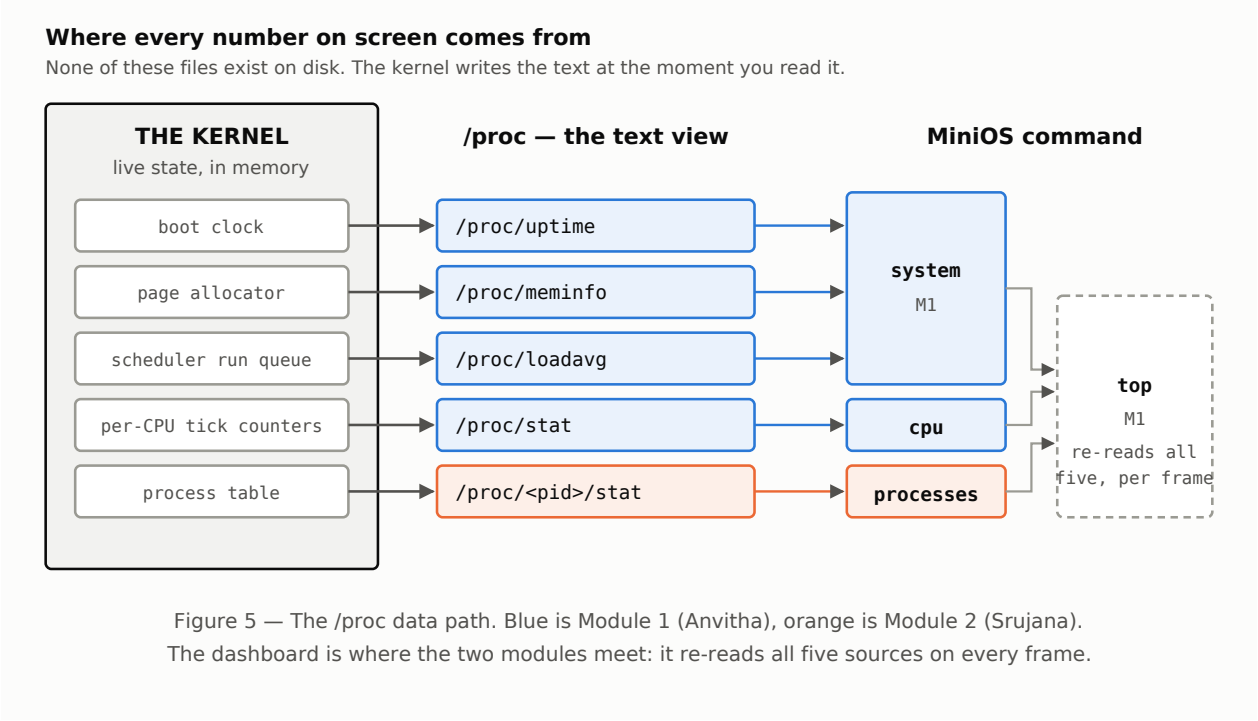


Figure 5 — The /proc data path. Blue is Module 1 (Anvitha), orange is Module 2 (Srujana). The dashboard is where the two modules meet: it re-reads all five sources on every frame.

3.2 What each file needed

Source	Format	How we parse it
`/proc/uptime`	two floats	one `fscanf("%lf %lf")`
`/proc/meminfo`	`Key: value kB` per line	`fscanf("%63[^:]: %lu")`, keep 7 keys
`/proc/loadavg`	`0.22 0.10 0.02 1/99 1806`	one `fscanf` with `%d/%d`
`/proc/stat`	counters per CPU, since boot	`sscanf` per `cpu*` line

Two details that cost time:

- Keys absent on some kernels (`MemAvailable` on older ones) must not break the whole read. `read_meminfo` zeroes the struct first and returns success if *any* key was found.
- `/proc/uptime`'s idle figure is summed across all CPUs, so it routinely exceeds the uptime. We label it rather than "correct" it.

3.3 The extensions, and why they are not padding

The handbook warns: *"Do not make MiniOS huge… Which OS concept does this feature teach?"* Both extensions have an answer.

cpu teaches that monotonic counters are meaningless alone. `/proc/stat` counts clock ticks since boot. The usage *right now* only exists as a difference between two readings:

```
busy% = 100 * (Δtotal − Δidle) / Δtotal
```

`idle` includes `iowait`, because the CPU had nothing to run then either. Implemented at `src/cpustat.c:77`.

`top` teaches three mechanisms at once: ANSI escape sequences to redraw in place, `termios` raw mode so a keypress arrives without Enter, and `select()` to wait on *either* the refresh timer *or* stdin. The last is the interesting one — a naive loop would either ignore the keyboard or busy-wait. The terminal settings belong to the user, so `term_restore()` runs on every exit path.

4. Module 2 — creating processes

4.1 The three calls

```
MiniOS> run date

  fork()      a second process exists; no program was loaded
  exec()      that process becomes 'date'; no process was created
  waitpid()   the parent collects the exit status
```

Implemented in `run_program()`, `src/process.c:39–66`. The handbook marks three TODOs there; all three are filled.

The separation matters: the window between `fork` and `exec` is the only place a child can change what the new program inherits — redirection, closing descriptors, resource limits. A combined call would have nowhere to put that.

4.2 Three defensive details

Line	What	Why
`process.c:39`	`fflush(NULL)` before `fork`	otherwise buffered output is duplicated into the child — measured in Part II Experiment 5
`process.c:50`	`_exit(127)` not `exit`	`exit` would flush the buffers inherited from the parent; 127 is the “not found” convention
`process.c:55`	`waitpid` in a retry loop	a signal returns `EINTR`, which means retry, not fail

4.3 Parsing `/proc/<pid>/stat`

52 fields on one line; we use six. The trap is field 2, the executable name, which is bracketed and may itself contain spaces and brackets — `(Web Content)` is a real process name. Splitting on whitespace, or on the first `)`, breaks. We search for the **last** `)` (`process.c:100`) and read numbers after it.

RSS is reported in **pages**, not bytes, so it is scaled by `sysconf(_SC_PAGESIZE)` rather than a hardcoded 4096 (`process.c:124`).

5. Integration

Covered fully in Part I. In brief: one shared header, and a list of command tables that the dispatcher walks in order. Merging the two modules required two lines of real change plus the resolution of three conflicts (a duplicated `proc_list`, five handlers whose source was not handed over, and one symbol defined in the wrong file).

The clearest evidence that the merge is real rather than cosmetic: M1's `top` calls M2's `proc_list()` every frame. M1's own copy of that code was deleted so there is exactly one owner.

6. Testing

6.1 Method

A shell suite, `tests/run_all.sh`, runs the actual binary and matches its output against an expected pattern. Three groups:

- **per module** — every command in the manual, with both valid and invalid arguments
- **integration** — three checks aimed at the seam itself: that `top` reaches M2's code, that both tables are reachable from `help`, and that a script mixing M1 and M2 commands runs
- **examples** — the four handbook demo programs still compile and behave

6.2 Result

```
34 passed, 0 failed
```

from a clean `make clean && make`, with zero compiler warnings under `-std=c11 -O2 -g -Wall -Wextra`.

6.3 Two bugs the suite caught

A flaky `top` test. `top` shows the busiest 8 processes, and on an idle machine MiniOS itself was sometimes rank 9 — so the test passed or failed depending on load. The fix was in the program, not the test: MiniOS's own row is now pinned regardless of rank, which is also better behaviour for a user.

A test that could not fail. The "blank line is harmless" check used a pattern that matched empty output, so it passed whatever happened. It was replaced with an explicit assertion that the output is empty *and* the exit code is 0. A test that cannot fail is worse than no test.

7. Findings

Full detail and method in Part II. The three that changed how we think:

exec is the expensive half. Making a process costs ~0.13 ms; loading a program into it costs ~1.0 ms, roughly 9x more. The intuition that "creating a process is expensive" is backwards — creation is cheap, loading is not.

fork does not copy memory. Holding 256 MB and forking costs 5.5 ms if the child exits immediately, and 184 ms if the child writes to every page — 34x more. Pages are shared read-only and copied one at a time on first write. This is precisely why `fork` followed immediately by `exec` is not the waste it appears to be.

fork copies the stdio buffer. A program with one `printf` before `fork` prints it twice when its output is piped, and once when it is on a terminal. Buffering mode, not the program, decides. This is a real bug class and the reason for `fflush(NULL)` in `run_program`.

A fourth, about method rather than operating systems: the first version of the copy-on-write experiment measured nothing, because `gcc -O2` deleted the child's write loop as a dead store. The result looked plausible — 256 MB appearing *faster* than 1 MB — which is the dangerous kind of wrong. `volatile` fixed it. We kept the episode in the report because the lesson generalises: **an optimiser can delete the thing you are trying to measure.**

8. Limitations and what remains

Not built. Modules 3–5 — pipes (Chapter 5), threads and race conditions (Chapters 6–7), semaphores and the bounded-buffer queue (Chapter 7) — and the optional xv6 system call (Chapter 8, Appendix A).

Partly handed over. Module 2's own extensions (`ps`, `wait`, `kill`, `runlimit`, `limit`) were listed in its command table but their source files were not provided. The five rows are commented out with a four-step restore note rather than deleted.

Experiments stop at Chapter 4. The handbook's week 8 is "producer-consumer and performance", which is experiment material we cannot produce without Modules 4 and 5.

Known rough edges.

- `processes` allocates a fixed 8192-entry array; a machine with more processes would be silently truncated.
- `top` re-reads the entire process table every frame rather than tracking changes. Measured at 0.413 ms for 64 processes, so it does not matter here, but it is linear and would at 2000.
- Thermal readings are untested on this machine — it has no sensors. The zero-sensor path is exercised; the reading path is not.

What the next merge needs. Nothing structural. A new module is one table in `minios.h`, one line in `shell.c`'s table list, and its `.c` file dropped into `src/`. The Makefile already globs. Part I section 9 has the five-step procedure.

9. References

1. Course handbook — *Operating Systems: MiniOS, A Learn → Run → Understand → Modify → Build Project*, Version 4 Revised. Chapters 3, 4, 10, 12, 13.

2. `proc(5)` man page — the format of every file under `/proc`, particularly the 52 fields of `/proc/<pid>/stat`.
3. `fork(2)`, `execve(2)`, `wait(2)`, `select(2)`, `termios(3)` man pages.
4. All measurements in Part II were produced by the programs in `experiments/`; raw output is in `docs/results.txt`.

Part V — Individual Viva Preparation

The handbook's own checklist and examiner questions, answered. Graded component: 10%.

The handbook is explicit that this is individual, and that siloing is not allowed:

> Group work does not mean that one person can own processes, another pipes, another threads and nobody else learns them.

So this covers what we built **and** gives an honest position on what we have not built yet.

A. The handbook's 13-item checklist

Items 1–5 and 12 are covered by Modules 1 and 2. Items 6–11 and 13 belong to Modules 3–5, which have not been handed over; honest answers for those are in section D.

1. I can explain /proc in simple words

`/proc` looks like a folder of files but nothing is on disk. The kernel generates the text at the moment you read it. It is a **view onto kernel data structures, dressed up as files** — which means ordinary tools (`cat`) and ordinary C (`fopen`, `fscanf`) can inspect the running system with no special API.

If pushed: numbered directories are one per process. `/proc/self` is whoever is asking. The files have size 0 because there is nothing stored until you read.

Show: `src/monitor.c:19` — `fopen("/proc/uptime")`, then `fscanf`.

2. I can run and explain a fork() program

```
./build/ex_fork_demo
```

`fork()` is called once and returns twice, because after it there are two processes. Both continue from the same line.

3. I can explain the parent/child return values from fork()

where	fork() returns	why
parent	the child's PID	the parent may have many children; it needs to know which one
child	0	the child has only one parent and can always find it with `getppid()`
failure	−1	no child was made; check `errno`

The asymmetry is the point: it is the **only** way the two processes can tell each other apart, since everything else about them is identical.

4. I can explain why parent and child do not automatically share ordinary variables

Each process has its own address space. After `fork()` the child has its own copy of the variable, so writing it in one does not affect the other.

The twist, which the handbook flags as "weird": both can print the *same address* while holding *different values*. Those are **virtual** addresses. Each process has its own page table, so the same virtual address maps to a different physical page.

The part we measured (Experiment 2): the copy does not happen at `fork()`. Pages are shared read-only, and the first write triggers a fault that copies that one page. Copy-on-write. We measured forcing the copy at **33.6x** the cost of not forcing it.

Show: `./build/ex_fork_memory_demo`, then Part II Experiment 2.

5. I can explain fork → exec → wait

```
MiniOS> run date

MiniOS
  |
  +-- fork()            now there are two of us
       |
       +-- parent ----- waitpid()   blocks until the child is done
       |
       +-- child ------ execvp("date")   stops being MiniOS,
                                         becomes date
```

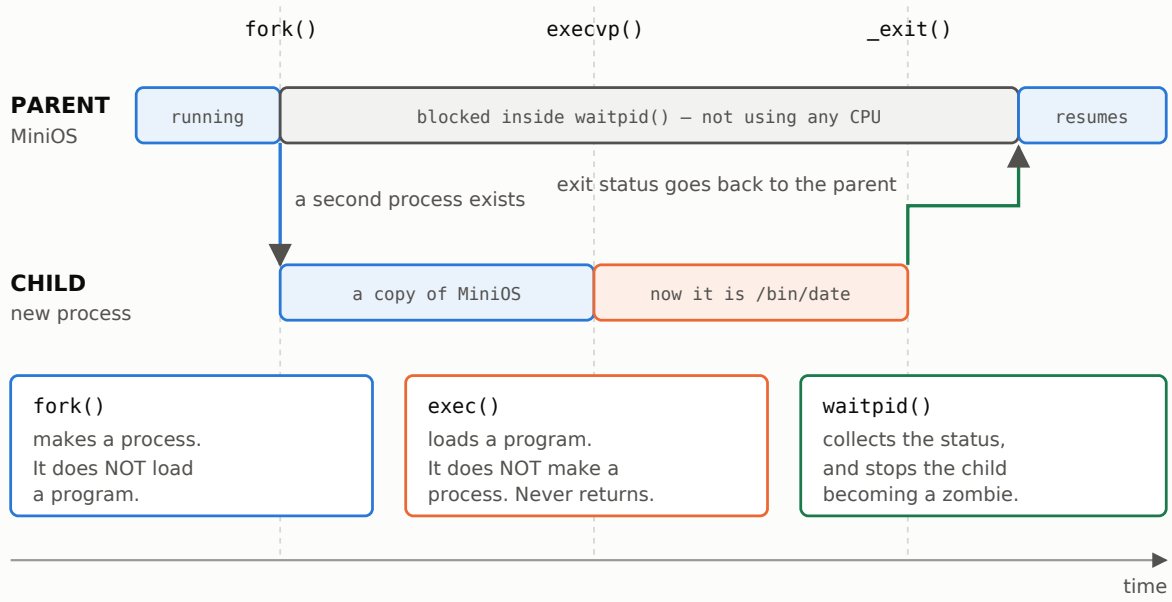


Figure 3 — Why fork and exec are separate calls: the gap between them is the only place a child can change what the new program will inherit (redirection, limits, open files).

Three separate jobs:

- `fork()` makes a process. It does **not** load a program.
- `exec()` loads a program. It does **not** make a process — it replaces the one calling it. On success it never returns.
- `waitpid()` lets the parent collect the exit status and stops the child becoming a zombie.

Why separate? Because the gap between fork and exec is where the child can change things that the new program will inherit — redirect stdout, close file descriptors, set resource limits. A single combined call would leave nowhere to do that. This is exactly where shell redirection lives.

Show: `src/process.c:39–66`.

12. I can explain how the MiniOS components fit together

```
src/shell.c                src/process.c
     |                           |
     |    both #include          |
     +-----------+-------------+
                 v
          include/minios.h
```

`minios.h` declares; each `.c` implements; no `.c` includes another. The dispatcher walks a list of command tables, one per module (`src/shell.c:45`). Adding a module is one line in that list.

The modules do genuinely touch: M1's `top` calls M2's `proc_list()` once per frame. That is the integration working, not a workaround.

Show: Part I sections 2 and 3.

B. The handbook's examiner questions

Answers kept to what an examiner will accept out loud.

Why does fork() return different values? So the two identical processes can tell which one they are. See A.3.

Why does the child not start executing at main()? Because `fork()` does not start a program — it duplicates one already running. The child resumes at the instruction after `fork()`, with the same program counter, stack and open files. `main()` is only entered when a program is *loaded*, which is `exec`'s job, not `fork`'s.

Why are their ordinary variables independent? See A.4 — separate address spaces, copy deferred by copy-on-write.

Why does the child use exec()? Because the child is still a copy of MiniOS, and we wanted `date`. `exec` replaces the program while keeping the process.

Why does the parent wait? Two reasons. To get the exit status, which is otherwise lost. And to reap the child — without `wait`, the finished child stays as a zombie holding a PID table slot. Demonstrated in Part II Experiment 3.

Why must unused pipe ends be closed? *(M3 — not built)* A reader gets EOF only when **every** write end is closed. If the reader still holds its own copy of the write end, it waits forever for itself. See section D.

What do threads share? / Why is count++ unsafe? / What is the critical section? / What does each semaphore represent? *(M4, M5 — not built)* See section D.

What is the path of a system call in xv6? *(optional, not built)* See section D.

C. Questions about our actual code

The examiner has the source open. These are the lines worth being ready for.

`run_program` calls `fflush(NULL)` before `fork()`. Why? `src/process.c:39`. Buffered output that has not been flushed is *in the buffer*, and `fork()` copies the buffer. Both processes then flush it, so the line prints twice. Only visible when output is redirected, because a terminal is line-buffered and a pipe is not. We measured it — Part II Experiment 5.

Why `_exit(127)` and not `exit(127)` when exec fails? `src/process.c:50`. `exit()` flushes stdio and runs `atexit` handlers. The child inherited copies of the parent's buffers, so flushing them would duplicate the parent's pending output. `_exit()` ends the process without touching them. 127 is the shell convention for "command not found".

`waitpid` is in a `while` loop. Why? `src/process.c:55`. A signal can interrupt a blocking call, which returns −1 with `errno == EINTR`. That is not a failure — it means "try again". Without the loop, pressing Ctrl-C during `run` would abandon the child.

Why search for the LAST `)` in `/proc/<pid>/stat`? `src/process.c:100`. Field 2 is the executable name in brackets, and that name can itself contain spaces and brackets — `(Web Content)` is real. Splitting on whitespace or the first `)` breaks on those. Searching backwards is safe because every field after it is a number.

Why multiply RSS by `sysconf(_SC_PAGESIZE)/1024`? `src/process.c:124`. The kernel reports RSS in **pages**, not bytes. Page size is 4096 here but is not guaranteed, so we ask rather than hardcode.

Explain the CPU percentage formula. `src/cpustat.c:77–85`.

```
busy% = 100 * (change in total − change in idle) / change in total
```

`/proc/stat` holds counters that only ever increase, since boot. A single reading tells you the average since boot, which is useless. Two readings and a subtraction tell you what happened in between. `idle` includes `iowait` because the CPU had nothing to run then either.

Why is `minios_interrupted` declared `volatile sig_atomic_t`? `include/minios.h`, set at `src/shell.c:131`. `volatile` stops the compiler caching it in a register — the loop would never see the change. `sig_atomic_t` guarantees the write is not torn halfway by a signal.

Why no `SA_RESTART` on the SIGINT handler? `src/shell.c:140`. With it, `fgets` would silently resume after Ctrl-C and the user would see nothing happen. Without it, `fgets` returns with `EINTR`, we print a newline and give a fresh prompt — which is what a shell should do.

`cmp_rows` compares doubles without casting to int. Why the fuss? `src/dashboard.c:70`. `return (int)(x - y)` on doubles truncates: 0.4−0.3 becomes 0 and the sort silently does nothing. We compare and return −1/0/1 instead.

Why does `top` restore the terminal on every exit path? `src/dashboard.c:60`. We turned off `ICANON` and `ECHO` in the user's terminal. Those settings belong to the terminal, not to us — if we exit without restoring them, the user's shell is left with no echo and they have to blind-type `reset`.

D. The modules we have not built — honest answers

Do not bluff these. The correct move is a short accurate answer plus "that is Module N, which is not merged yet."

Pipes (M3). A pipe is a one-way byte stream in kernel memory with two file descriptors — `fd[0]` to read, `fd[1]` to write. It carries bytes, not messages, so a reader can get half of what was written and must frame its own records. The reader sees EOF only when every copy of the write end is closed, which is why each side closes the end it does not use.

Threads (M4). Threads of one process share the address space — globals and the heap — so a pointer means the same thing in all of them. Each has its own stack and registers. That sharing is the point and the danger.

Why `count++` is unsafe (M4). It is three machine operations: load, add, store. Two threads can both load 5, both store 6, and one increment disappears. The **critical section** is the stretch of code that must not be interleaved — here, those three operations.

Semaphores in a bounded buffer (M5). Three, with distinct jobs: `empty` counts free slots (producer waits on it), `full` counts items (consumer waits on it), `mutex` protects the buffer structure itself. Count semaphores are waited on **before** the mutex — the other order deadlocks, because a thread would sleep holding the lock everyone needs.

Full vs empty queue (M5). They are different conditions with different waiters: full blocks producers, empty blocks consumers. One flag cannot represent both, which is why there are two counting semaphores.

xv6 syscall path (optional). User program calls the stub in `usys.S`, which puts the syscall number in register `a7` and executes `ecall`. The trap enters `uservec`, saves the user registers, switches to the kernel page table, calls `syscall()`, which looks the number up in a table and calls the handler. The return value goes back in `a0`, then `sret` returns to user mode.

E. Two-minute self-test before you walk in

Say each of these out loud. If you hesitate, reread the section.

```
[ ] /proc is the kernel answering, not files on disk
[ ] fork returns twice; child 0, parent the PID, -1 on failure
[ ] fork makes a process, exec loads a program, wait collects the status
[ ] the gap between fork and exec is where redirection happens
[ ] same address, different value = virtual memory + copy-on-write
[ ] zombie = finished but not yet waited for; leaks PID slots
[ ] orphan = parent died first; reparented to PID 1
[ ] one /proc/stat reading is useless; you need two and a subtraction
[ ] fflush(NULL) before fork, or buffered output doubles
[ ] the modules meet in one header and one table list
```

F. If something goes wrong in the viva

- **You do not know.** Say so, then say what you would do to find out (`man 2 fork`, read the source, write a test). The handbook's own tone invites this: *"You are allowed to be confused."*
- **You are asked about a teammate's module.** Give the section D answer, then say plainly it is not merged yet.
- **The examiner finds a bug.** Agree if they are right. Say how you would test the fix. Defending a real bug costs more than the bug does.
- **A demo command fails.** `./minios demo.txt` runs everything non-interactively. Part III section 6 has the per-command fallbacks.